

Copilot + Claude

Sonnet 4.6

Why It Matters for the Risk Analyst's Daily Job

From Oracle queries to MCP, from Excel chaos to Python + Prefect: a productivity playbook.

What this document covers

- ✓ Why Claude Sonnet 4.6 inside Copilot is a fit for risk analysis
- ✓ Oracle SQL queries vs. MCP SQL Developer service
- ✓ Letting AI read Excel, XLLs and DLLs to understand legacy projects
- ✓ Building reusable SQL & Python assets cheaper models can run
- ✓ Migrating manual Excel workflows to Python + Prefect pipelines

Executive Summary

The risk analyst's day is rarely about "one big model." It is about hundreds of small frictions: an Oracle query rewritten for the fifth time, an Excel sheet someone built in 2014 that nobody fully understands, a manual reconciliation kicked off every morning. Each friction is small. The aggregate is enormous.

GitHub Copilot powered by **Claude Sonnet 4.6** is not a magic button — it is a high-quality *pair* that sits inside the analyst's IDE and removes precisely those frictions. This document explains, with concrete patterns, why pairing Copilot with Claude Sonnet 4.6 is a strong fit for the risk analyst role at CACIB, and how it changes four day-to-day workflows: SQL data access, legacy code archaeology, reusable-asset creation, and migration of brittle Excel processes to production-grade Python pipelines orchestrated by Prefect.

💡 Core thesis

The value is not in Claude writing code faster than the analyst. The value is in Claude reading, explaining, and **standardising** the messy artefacts (SQL, Excel, XLL, DLL, manual SOPs) the analyst inherits — producing portable assets that weaker models, or zero-cost manual runs, can execute downstream.

1 Why Claude Sonnet 4.6 inside Copilot fits Risk Analysis

A risk analyst is, on most days, a translator: business questions → data extracts, regulatory text → control logic, legacy spreadsheets → reproducible numbers. Claude Sonnet 4.6 is well-positioned for this role for three reasons.

1.1 Long context, structured reasoning, low fabrication on data tasks

Sonnet 4.6 maintains coherence across long files — a 4,000-line Oracle PL/SQL package, an Excel workbook exported to CSV, a 60-page model methodology — without losing the thread. For a risk analyst this matters because the truth about a number almost never lives in one cell; it lives in a chain of joins, lookups, and assumptions.

1.2 Inside the IDE, not in a browser tab

Copilot brings the model where the work happens: Visual Studio, VS Code, and Visual Studio Code Insiders. The analyst doesn't paste code into a chat window and lose the workspace context. The model sees the open file, the referenced project, the test, and the data shape.

1.3 It produces *assets*, not just answers

This is the critical point. A good interaction with Sonnet 4.6 ends with a committed file: a vetted SQL view, a documented Python module, a Prefect flow, a markdown SOP. Those assets then serve the team — and often serve cheaper models too, which can re-run or lightly adapt them at a fraction of the cost.

A note on the project skeleton

This document is aligned with the folder layout you started under C:\Test\risk-analyst-copilot. The four work-streams below map naturally to that structure: sql/, legacy-analysis/, shared-scripts/, and migrations/.

2 Oracle SQL queries vs. the MCP SQL Developer service

Risk analysts at CACIB live in Oracle. The question is not *whether* to use Oracle — it is *how* the AI assistant talks to it.

2.1 The “copy-paste” baseline

The default workflow is: open SQL Developer, write a query, copy the result back to Excel, paste into Copilot chat for explanation. This works but has well-known costs.

Dimension	Oracle SQL Developer (copy-paste)	MCP SQL Dev service
Schema awareness	Manual — analyst pastes DDL	✓ Live introspection
Query iteration	Run, copy error, paste back	✓ Run & self-correct loop
Data-shape grounding	Model guesses column types	✓ Model sees real types & samples
Lineage / dependency	Lost	Can trace views & synonyms
Reproducibility	Ad-hoc in a chat window	✓ Saved as SQL files in repo
Data exposure	Pastes raw rows into prompt	Controlled — only what’s needed

2.2 What “MCP SQL Dev service” actually changes

The Model Context Protocol gives the model a *capability*, not just text. Instead of guessing at the schema, Claude can call a tool that says “describe RISK_POSITIONS”, “sample 5 rows from COUNTERPARTY_REF”, or “run this query and return the first 20 rows.”

🔧 Practical effect

A query that took three iterations in the copy-paste workflow typically takes one. The model sees the actual column names (CPTY_ID vs. COUNTERPARTY_ID), the actual NULL patterns, and the actual cardinality before writing the first JOIN.

2.3 Side-by-side example

Analyst request: “Give me the top 20 counterparties by aggregated exposure as of last business day, broken down by product family, with a sanity check on negative exposures.”

Without MCP (model guesses):

```
-- Model guesses schema. Often wrong on first try.
SELECT cpty_name, product_family, SUM(exposure) AS exp
FROM risk_positions
WHERE business_date = TRUNC(SYSDATE) - 1
GROUP BY cpty_name, product_family
ORDER BY exp DESC
FETCH FIRST 20 ROWS ONLY;
-- Probable errors: column may be EXPOSURE_AMT; date logic
-- ignores weekends; no NULL guard; no negative-exposure check.
```

With MCP (model introspects):

```
WITH last_bd AS (
  SELECT MAX(business_date) AS bd
  FROM risk_positions
  WHERE business_date <= TRUNC(SYSDATE)
),
agg AS (
  SELECT c.cpty_legal_name,
         p.product_family,
         SUM(NVL(rp.exposure_amt, 0))           AS exposure_total,
         SUM(CASE WHEN rp.exposure_amt < 0
                  THEN rp.exposure_amt ELSE 0 END) AS exposure_negative
  FROM   risk_positions rp
  JOIN   counterparty_ref c ON c.cpty_id = rp.cpty_id
  JOIN   product_ref      p ON p.product_id = rp.product_id
  JOIN   last_bd          d ON d.bd = rp.business_date
  GROUP BY c.cpty_legal_name, p.product_family
)
SELECT *
FROM   agg
ORDER BY exposure_total DESC
FETCH FIRST 20 ROWS ONLY;
```

The second version is not just longer — it reflects *real* column names and a real “last business day” definition. That difference comes from the MCP tool call, not from a better prompt.

3 From conversation to reusable, shippable assets

This is the section that pays for the licence. The point of a Claude Sonnet 4.6 interaction is rarely the answer in the chat window — it is the file that lands in the repo.

3.1 Common behaviours as “ship-ready” SQL

A risk team accumulates patterns: “exposure as of last business day,” “netting set membership,” “counterparty hierarchy roll-up,” “haircut applied to collateral.” Each of these gets re-written by a different analyst every quarter. Sonnet 4.6 is well-suited to turning each pattern into a parameterised, documented view that the whole team — and weaker models downstream — can simply call.

```
-- shared-scripts/sql/views/v_exposure_last_bd.sql
-- Owner: Risk Analytics   Reviewed: yyyy-mm-dd
-- Purpose: Canonical "exposure as of last business day" view.
-- Consumers: weaker models, manual extracts, Prefect flows.
CREATE OR REPLACE VIEW v_exposure_last_bd AS
WITH last_bd AS (
  SELECT MAX(business_date) AS bd
  FROM   risk_positions
  WHERE  business_date <= TRUNC(SYSDATE)
)
SELECT rp.cpty_id,
       rp.product_id,
       rp.business_date,
       NVL(rp.exposure_amt, 0)           AS exposure_amt,
       NVL(rp.exposure_ccy, 'EUR')      AS exposure_ccy
FROM   risk_positions rp
JOIN   last_bd        d ON d.bd = rp.business_date;
```

The leverage point

Once `v_exposure_last_bd` exists, every downstream task — including the ones run by cheaper models or by hand — becomes a one-liner. The expensive model is paid once for the pattern, not every time the pattern is used.

3.2 Why this is good for weaker / cheaper models too

Smaller models often fail not because they “don’t know SQL” but because they don’t know *your* schema. A library of vetted views collapses the surface area: the small model only needs to know “SELECT ... FROM `v_exposure_last_bd` WHERE ...”. That is well within their reliable range — and it is also well within the range of an analyst running it by hand at zero marginal cost.

4 Reading Excel, XLLs and DLLs — archaeology made tractable

Every risk team has *the workbook*. The one that calls a custom XLL written by someone who left in 2017, which in turn loads a DLL whose source is on a network share that may or may not still exist. Today this is a multi-day investigation. With Sonnet 4.6 inside Copilot it becomes a guided walkthrough.

4.1 What the model can actually do

Artefact	What Claude Sonnet 4.6 can extract / explain
.xlsx workbooks	Sheet inventory, named ranges, formula dependency graph, identification of input cells vs. computed cells, hidden sheets, external links
.xlsm (with VBA)	Walks every Sub/Function; flags <code>Application.Run</code> cross-calls; documents implicit globals and <code>Workbook_Open</code> side-effects
.xll add-ins	With exported symbol list, infers UDF signatures and which workbook formulas depend on them
.dll (C/C++/C#)	With exported symbols (or PDB / decompiled C#), maps the external API surface the workbook relies on

4.2 A typical archaeology session

- Analyst opens the workbook in VS Code, exports a CSV of all formulas via a small Python script.
- Claude reads the CSV and produces a **dependency map**: which sheets feed which, where the inputs come from, which cells are dead code.
- Claude reads the VBA modules and writes a `LEGACY-NOTES.md` describing the macros in plain English, with a **flag for any logic that no longer matches current methodology**.
- For the XLL/DLL boundary, Claude lists the imported functions and proposes Python equivalents (e.g. a `scipy.optimize` call for a bespoke solver UDF).

Caveat worth stating clearly

Pure binary .dll reverse engineering without symbols or source is bounded. Claude is excellent at reasoning over exports, headers, PDBs, decompiled IL (for .NET DLLs), and any accompanying documentation. It does not magically read stripped native binaries. The realistic value: turn a 3-week archaeology dig into a 2-day guided walkthrough.

5 Migrating Excel pipelines to Python + Prefect

This is where the day-to-day burden actually lifts. Most risk teams have at least one “every-morning-at-7am” workbook: open, refresh, paste, save-as, e-mail. Migrating it to Python orchestrated by Prefect removes the human from the loop while preserving the calculations the business has signed off on.

5.1 The migration pattern



5.2 Concrete example: the daily exposure pack

Before. A workbook pulls from Oracle via an ODBC query, applies a few pivot tables, runs a VBA macro to colour cells, saves a PDF, and the analyst e-mails it at 07:30. Total: 25 minutes of human time, every day, with a non-trivial error rate when the analyst is on leave.

After (target architecture).

```

# migrations/daily_exposure_pack/flow.py
from prefect import flow, task
from prefect.schedules import CronSchedule
import pandas as pd
from .sql import fetch_exposure_last_bd
from .render import build_pdf_pack
from .notify import send_email

@task(retries=2, retry_delay_seconds=120)
def extract() -> pd.DataFrame:
    return fetch_exposure_last_bd() # uses v_exposure_last_bd view

@task
def transform(df: pd.DataFrame) -> pd.DataFrame:
    pivot = (df.groupby(["product_family", "cpty_legal_name"])
             ["exposure_amt"].sum()
             .unstack(fill_value=0))
    return pivot

@task
def render(pivot: pd.DataFrame) -> str:
    return build_pdf_pack(pivot, title="Daily Exposure Pack")

@task
def deliver(pdf_path: str) -> None:
    send_email(to="risk-mgmt@cacib.com",
              subject="Daily Exposure Pack",
              attachments=[pdf_path])

@flow(name="daily-exposure-pack")
def daily_exposure_pack():
    df = extract()
    pivot = transform(df)
    pdf = render(pivot)
    deliver(pdf)

if __name__ == "__main__":
    daily_exposure_pack.serve(
        name="prod",
        schedule=CronSchedule(cron="30 7 * * 1-5", timezone="Europe/Paris")
    )
  
```

5.3 What Claude Sonnet 4.6 actually did during this migration

- Read the legacy workbook and produced the plain-English spec.
- Wrote the Oracle query, validated it against the live schema (MCP).
- Wrote the Python module *and* its unit tests, including edge cases on negative exposure and missing FX rates.
- Generated the Prefect flow with sensible retries, scheduling, and a deployment manifest.
- Wrote the runbook (RUNBOOK.md) for L1 support.

Net effect on the analyst's day

The 25 daily minutes are gone. More importantly, the workbook stops being a single point of failure tied to one person's laptop. The pipeline now lives in version control, runs on schedule, is monitored, and is something a junior or a weaker model can extend.

6 Proposed project skeleton

A folder layout that operationalises everything above — aligned with what you started under `C:\Test\risk-analyst-`

```
risk-analyst-copilot/
|-- README.md
|-- .github/
|   '-- copilot-instructions.md      # House rules for the model
|-- sql/
|   |-- views/                      # Vetted, shippable views
|   |-- snippets/                  # Common WHERE/JOIN patterns
|   '-- queries/                    # One-off, dated
|-- legacy-analysis/
|   |-- workbooks/                  # Copies of legacy .xlsx/.xlsm
|   |-- notes/                      # LEGACY-NOTES.md per workbook
|   '-- dll-xll/                    # Symbol dumps, header files
|-- shared-scripts/
|   |-- python/                     # Reusable utilities
|   '-- sql/                        # Linked to sql/views/
|-- migrations/
|   '-- daily_exposure_pack/         # One folder per migrated workbook
|       |-- flow.py
|       |-- sql.py
|       |-- render.py
|       |-- tests/
|       '-- RUNBOOK.md
'-- docs/
    '-- prompting/                  # Reusable prompts for the team
```

6.1 One file worth highlighting: copilot-instructions.md

A short house-rules file in `.github/` lets every analyst on the team get consistent behaviour from Copilot: “always use `v_*` views when they exist,” “always write a unit test alongside any pricing function,” “never hardcode FX rates,” “flag any logic that diverges from the methodology doc.” This is how the team’s standards become the model’s defaults.

7 Risks, costs and what to watch

Risk	Mitigation
Data leaving the bank’s perimeter	Enterprise Copilot tenancy; never paste production data into ad-hoc chats; rely on MCP for controlled access
Model confidently wrong on niche methodology	Pair every generated file with unit tests; require human sign-off on anything entering shared-scripts/
Over-reliance on the expensive model	Use Sonnet 4.6 to <i>produce</i> assets; let cheaper models or manual runs <i>consume</i> them
“Shadow IT” migrations bypassing controls	All Prefect flows go through the team’s standard CI / review process; no flow without a runbook

8 Recommendation

Adopt Copilot + Claude Sonnet 4.6 as the analyst’s default IDE assistant, with three explicit work-streams running in parallel for the first quarter:

- SQL standardisation.** Stand up the MCP SQL Dev service in a controlled environment. Build out `sql/views/` with the ten most-requested risk patterns.
- Legacy archaeology.** Pick the three most critical legacy workbooks. Produce `LEGACY-NOTES.md` for each. Do not migrate yet — understand first.
- First migration.** Migrate one workbook end-to-end to Python + Prefect. Measure the time saved and the error reduction. Use the result as the template for the next ten.

The expensive model is paid *once* for each pattern. The pattern then serves the team — and the cheaper models, and the manual fallbacks — for as long as it lives in the repo. That is where the productivity actually compounds.